

DA3408 - Assignment 1

K. Sree Hasini DA24B041

August 2026

Question 1

(a) **Changing Anything Changes Everything (CACE):** The two features appear to be independent, but they are actually connected through the model. Changing one feature does not affect only its own parameters or behaviour; it can also affect the parameters and behaviour of other features in the model. Therefore, a change made to one part of the system can unexpectedly impact other seemingly unrelated parts.

(b) **Undeclared Consumers:** An example scenario to reason about this case is, let's say the marketing team announce some offers on the less favourite restaurants, about which is the ML team unaware. As the ML team doesn't know about the new prices, the performance of model predicting favourite restaurants decays.

(c) **Configuration & Glue Code Debt:** There is no clear way to know which script should run first or what configuration each script uses. This makes the pipeline difficult to understand, reproduce, and maintain. A new team member would find it difficult to understand and modify the existing code.

A practical solution is to follow the **Reproducibility Protocol**. The 14 scripts should be consolidated into a single, version-controlled entry point, such as a `train.py` file or a clearly documented `Makefile`/pipeline script. The complete training process should be committed to Git. This makes it possible to identify the exact commit that was used for a particular training run. Record the packages and versions used by the training pipeline through `environment.yml`.

Question 2

I chose `batch_size = [32, 128, 256]` and `learning_rate_init = [0.001, 0.005]` as the hyperparameter combinations for the experiments.

1. Best Performing Run: The best model is `batch_size = 32` and `learning_rate_init = 0.001` because it achieved the highest validation accuracy. Though there are other models with lower training loss, they performed relatively poorly in terms of generalisation on unseen data, i.e., the validation set. This is a sign of **overfitting**: lower training loss but lower validation accuracy.

2. Evidence of Overfitting: Higher batch sizes gave lower training loss, such as for `batch_size = 256`, `lr = 0.005` and `batch_size = 256`, `lr = 0.001`. However, their validation-set performance was not as good as the model with `batch_size = 32`, `lr = 0.001`, which had a higher training loss. Therefore, a lower training loss does not necessarily imply better generalisation.

3. Hyperparameter with Bigger Effect: Fixing `lr = 0.001` and changing batch size (`32 → 128 → 256`): `val_accuracy = 0.975 → 0.973 → 0.971`, which is a small change.

Fixing `lr = 0.005` and changing batch size: `val_accuracy = 0.965 → 0.972 → 0.972`; `batch_size = 32` is a noticeably worse outlier.

Fixing `batch_size = 32` and changing the learning rate (`0.001 → 0.005`): `val_accuracy = 0.975 → 0.965`, which is the biggest change. Identified overfitting through validation accuracy metric.

Question 3

- In this question, I first initialized DVC in my git repository and used the given dataset to create the first version (v1). I created an S3 bucket in AWS to store the DVC data and connected it to my local repository using AWS IAM credentials. I kept the credentials locally and did not commit them to github.
- For v1, I created a CSV containing the filenames of the images, which had 1800 filenames. I used `dvc add` to track this file and then used `dvc push` to store the crripdg data in the S3 bucket. I committed and tagged this version as v1.
- Next, I added the data from `new_labels.zip` to simulate a data update. This increased the number of image files to 2800, so the CSV had 2801 lines including the header. I ran `dvc add` again and committed this as v2.
- Finally, I tested whether I could go back to the older version of the dataset. I used `git checkout v1` followed by `dvc checkout`. I verified the rollback by checking the number of lines in the CSV using `wc -l`, which gave 1801 lines (1800 filenames plus the header). This confirmed that the v1 version of the dataset was successfully retrieved. After completing the test, I returned to the latest version using `git checkout main` and `dvc checkout`.

Question 4